

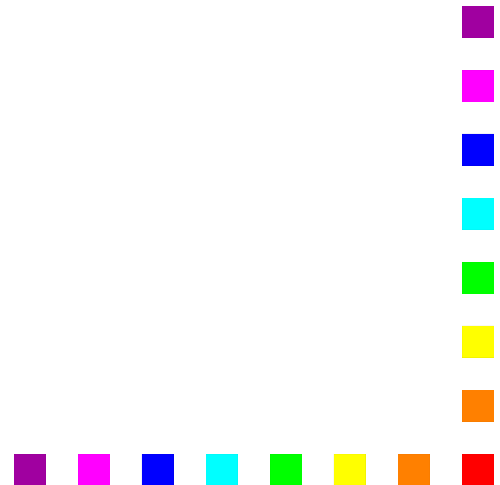
Introduction to Sequential Files

COBOL's forte

COBOL is generally used in situations where the volume of data to be processed is large.

These systems are sometimes referred to as “**data intensive**” systems.

Generally, large volumes of data arise **not** because the data is inherently voluminous but because the **same items** of information have been recorded about a **great many instances** of the same object.



Files, Records, Fields.

We use the term **FIELD** to describe an item of information we are recording about an object
(e.g. StudentName, DateOfEntry)

We use the term **RECORD** to describe the collection of fields which record information about an object
(e.g. a StudentRecord is a collection of fields which record information about a student)

We use the term **FILE** to describe a collection of one or more occurrences (instances) of a record type (template).

It is important to distinguish between the record occurrence (i.e. the values of a record) and the record type (i.e. the structure of the record).
Every record in a file has a **different value** but the **same structure**.



Files, Records, Fields.

STUDENTS.DAT

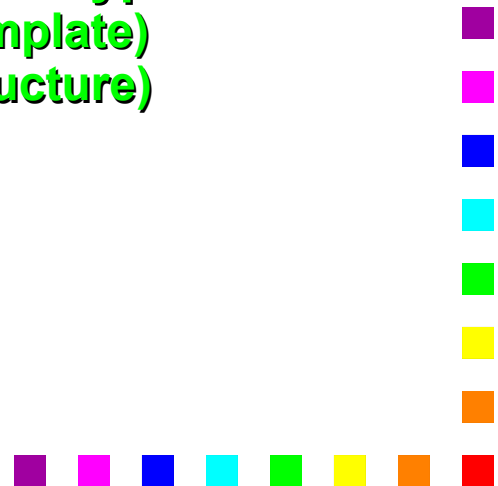
StudId	StudName	DateOfBirth
--------	----------	-------------

9723456	COUGHLAN	10091961
9724567	RYAN	31121976
9534118	COFFEY	23061964
9423458	O'BRIEN	03111979
9312876	SMITH	12121976

occurrences

```
DATA DIVISION.  
FILE SECTION.  
FD StudentFile.  
01 StudentDetails.  
    02  StudId      PIC 9(7) .  
    02  StudName    PIC X(8) .  
    02  DateOfBirth PIC X(8) .
```

Record Type
(Template)
(Structure)



How files are processed.

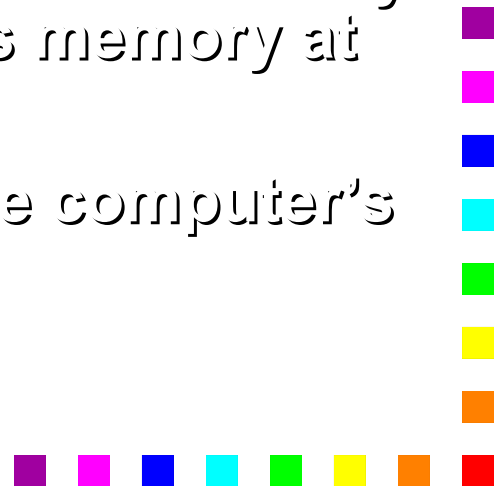
Files are repositories of data that reside on backing storage (hard disk or magnetic tape).

A file may consist of hundreds of thousands or even millions of records.

Suppose we want to keep information about all the TV license holders in the country. Suppose each record is about 150 characters/bytes long. If we estimate the number of licenses at 1 million this gives us a size for the file of $150 \times 1,000,000 = 150 \text{ megabytes}$.

If we want to process a file of this size we cannot do it by loading the whole file into the computer's memory at once.

Files are processed by reading them into the computer's memory **one record** at a time.



Record Buffers

To process a file records are read from the file into the computer's memory **one record at a time**.

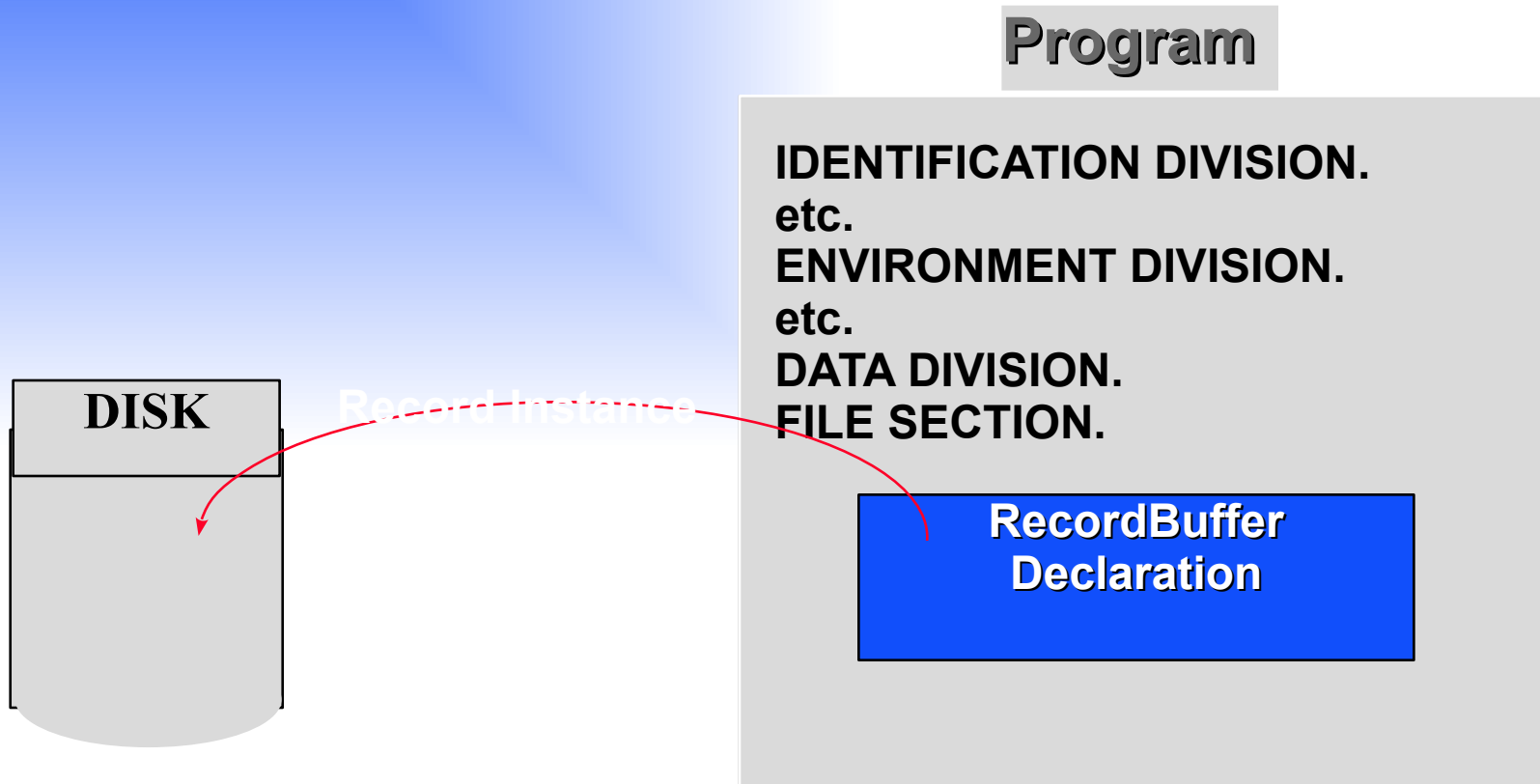
The computer uses the programmers description of the record (i.e. the record template) to set aside sufficient memory to store **one instance** of the record.

Memory allocated for storing a record is usually called a "**record buffer**"

The record buffer is the **only** connection between the program and the records in the file.



Record Buffers



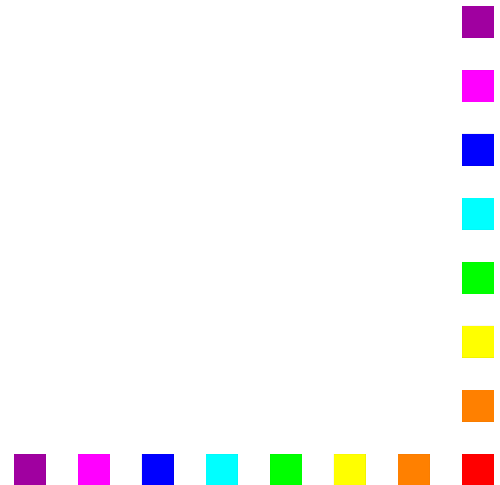
Implications of 'Buffers'

If your program processes more than one file you will have to describe a record buffer for **each** file.

To process all the records in an **INPUT file** each record instance must be copied (read) from the file into the record buffer when required.

To create an **OUTPUT file** containing data records each record must be placed in the record buffer and then transferred (written) to the file.

To transfer a record from an input file to an output file we will have to



Creating a Student Record

Student Details.

Student Id.

01 StudentDetails.

02 StudentId PIC 9(7).

Student Name.

02 StudentName.

Surname 03 Surname PIC X(8).

Initials 03 Initials PIC XX.

Date of Birth

02 DateOfBirth.

Year of Birth 03 YOBirth PIC 99.

Month of Birth 03 MOBirth PIC

99.

Day of Birth 03 DOBirth PIC 99.

Course Code

02 CourseCode PIC X(4).

Value of grant

02 Grant PIC 9(4).

Gender

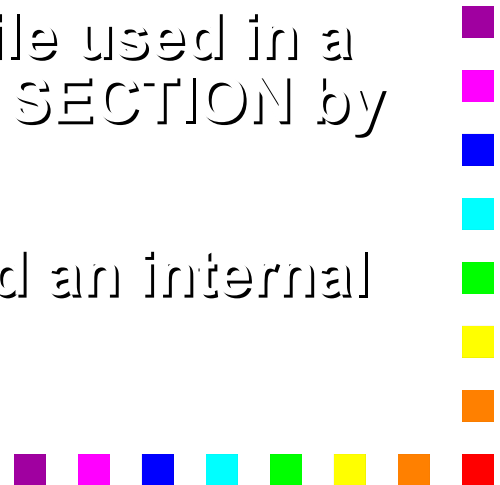
02 Gender PIC X.

Describing the record buffer in COBOL

```
DATA DIVISION.  
FILE SECTION.  
FD StudentFile.  
01 StudentDetails.  
    02 StudentId          PIC 9(7) .  
    02 StudentName.  
        03 Surname        PIC X(8) .  
        03 Initials        PIC XX .  
    02 DateOfBirth.  
        03 YOBirth         PIC 9(2) .  
        03 MOBirth         PIC 9(2) .  
        03 DOBirth         PIC 9(2) .  
    02 CourseCode         PIC X(4) .  
    02 Grant              PIC 9(4) .  
    02 Gender              PIC X .
```

The record type/template/buffer of **every** file used in a program **must** be described in the FILE SECTION by means of an FD (file description) entry.

The FD entry consists of the letters FD and an internal file name.



The Select and Assign Clause.



```
ENVIRONMENT DIVISION.  
INPUT-OUTPUT SECTION.  
FILE-CONTROL.  
    SELECT StudentFile  
        ASSIGN TO "STUDENTS.DAT".  
  
DATA DIVISION.  
FILE SECTION.  
FD StudentFile.  
01 StudentDetails.  
    02 StudentId          PIC 9(7) .  
    02 StudentName.  
        03 Surname       PIC X(8) .  
        03 Initials      PIC XX.  
    02 DateOfBirth.  
        03 YOBirth       PIC 9(2) .  
        03 MOBirth       PIC 9(2) .  
        03 DOBirth       PIC 9(2) .  
    02 CourseCode        PIC X(4) .  
    02 Grant              PIC 9(4) .  
    02 Gender             PIC X.
```

The internal file name used in the FD entry is connected to an external file (on disk or tape) by means of the Select and Assign clause.



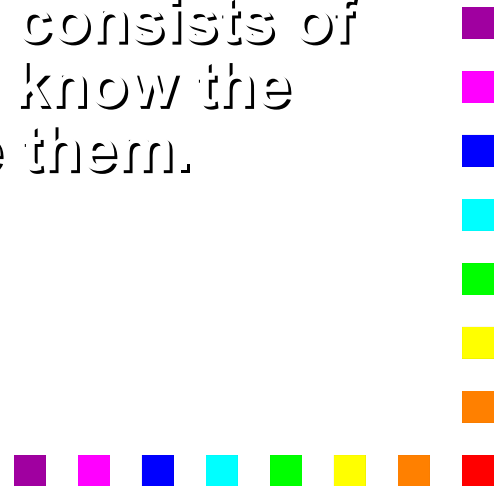
Select and Assign Syntax.

SELECT FileName ASSIGN TO ExternalFileReference

[ORGANIZATION IS $\left\{ \begin{array}{c} \underline{\text{LINE}} \\ \underline{\text{RECORD}} \end{array} \right\}$ SEQUENTIAL].

LINE SEQUENTIAL means each record is followed by the carriage return and line feed characters.

RECORD SEQUENTIAL means that the file consists of a stream of bytes. Only the fact that we know the size of each record allows us to retrieve them.



COBOL file handling Verbs

OPEN

Before your program can access the data in an input file or place data in an output file you must make the file available to the program by **OPEN**ing it.

READ

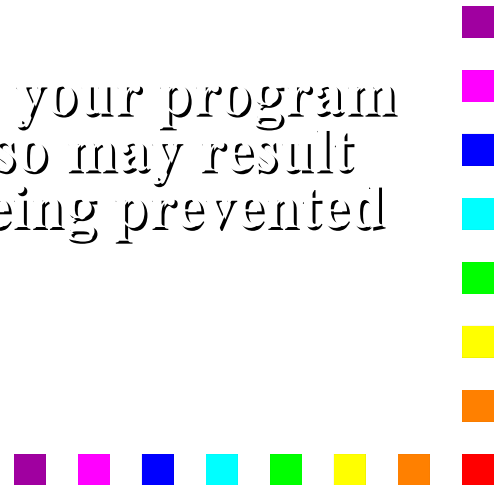
The **READ** copies a record occurrence/instance from the file and places it in the record buffer.

WRITE

The **WRITE** copies the record it finds in the record buffer to the file.

CLOSE

You must ensure that (before terminating) your program closes all the files it has opened. Failure to do so may result in data **not** being written to the file or users being prevented from accessing the file.



OPEN and CLOSE verb syntax

$$\underline{\text{OPEN}} \left\{ \left\{ \begin{array}{c} \underline{\text{INPUT}} \\ \underline{\text{OUTPUT}} \\ \underline{\text{EXTEND}} \end{array} \right\} \text{InternalFileName} \right\} \dots$$

When you open a file you have to indicate to the system what how you want to use it (e.g. INPUT, OUTPUT, EXTEND) so that the system can manage the file correctly.

Opening a file **does not transfer** any data to the record buffer, it simply provides access.



The READ verb

Once the system has opened a file and made it available to the program it is the programmers responsibility to process it correctly.

Remember, the file record buffer is our only connection with the file and it is only able to store a single record at a time.

To process all the records in the file we have to transfer them, one record at a time, from the file to the buffer.

COBOL provides the READ verb for this purpose.



READ verb syntax

```
READ InternalFilename [NEXT] RECORD  
      [INTO Identifier]  
      AT END StatementBlock  
END - READ
```

The InternalFilename specified must be a file that has been **OPEN**ed for INPUT.

The NEXT RECORD clause is optional and generally not used.

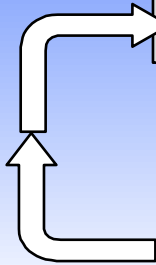
Using INTO Identifier clause causes the data to be read into the record buffer and then copied from there to the specified Identifier in one operation.



How the READ works

StudentRecord

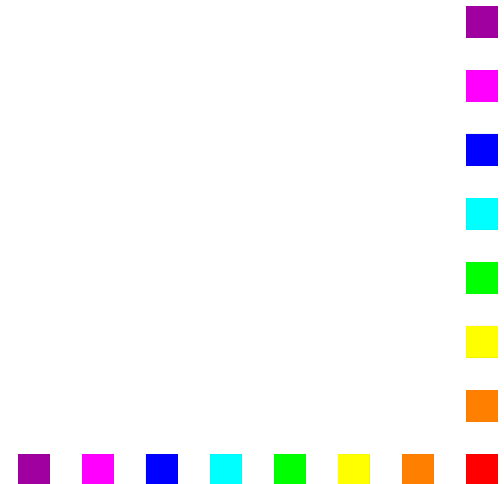
StudentID	StudentName	Course.
9 3 3 4 5 6 7	F r a n k C u r t a i n	L M 0 5 1



9	3	3	4	5	6	7	F	r	a	n	k		C	u	r	t	a	i	n			L	M	0	5	1
9	3	8	3	7	1	5	T	h	o	m	a	s		H	e	a	l	y				L	M	0	6	8
9	3	4	7	2	9	2	T	o	n	y		O	'	B	r	i	a	n				L	M	0	5	1
9	3	7	8	8	1	1	B	i	l	l	y		D	o	w	n	e	s				L	M	0	2	1

READ StudentRecords

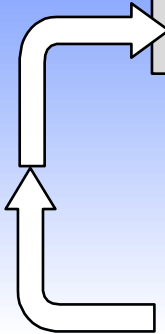
END-READ



How the READ works

StudentRecord

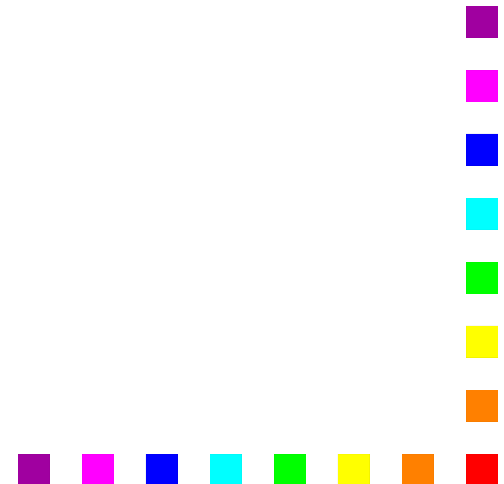
StudentID	StudentName	Course.
9 3 8 3 7 1 5	T h o m a s H e a l y	L M 0 6 8



9	3	3	4	5	6	7	F	r	a	n	k		C	u	r	t	a	i	n			L	M	0	5	1
9	3	8	3	7	1	5	T	h	o	m	a	s		H	e	a	l	y				L	M	0	6	8
9	3	4	7	2	9	2	T	o	n	y		O	'	B	r	i	a	n				L	M	0	5	1
9	3	7	8	8	1	1	B	i	l	l	y		D	o	w	n	e	s				L	M	0	2	1

READ StudentRecords

END-READ



How the READ works

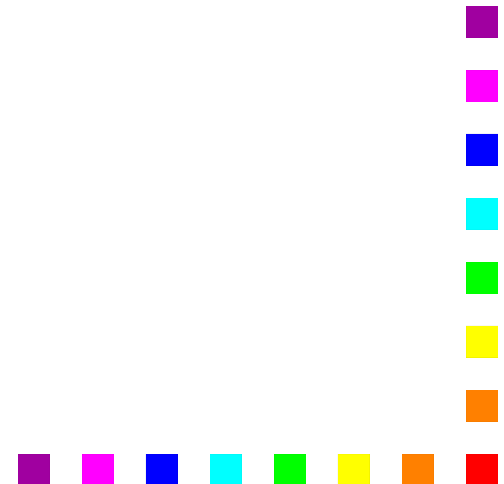
StudentRecord

StudentID	StudentName	Course.
9 3 4 7 2 9 2	T o n y O ' B r i a n	L M 0 5 1

9	3	3	4	5	6	7	F	r	a	n	k		C	u	r	t	a	i	n			L	M	0	5	1
9	3	8	3	7	1	5	T	h	o	m	a	s		H	e	a	l	y				L	M	0	6	8
9	3	4	7	2	9	2	T	o	n	y		O	'	B	r	i	a	n				L	M	0	5	1
9	3	7	8	8	1	1	B	i	l	l	y		D	o	w	n	e	s				L	M	0	2	1

READ StudentRecords

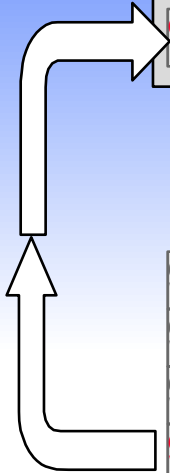
END-READ



How the READ works

StudentRecord

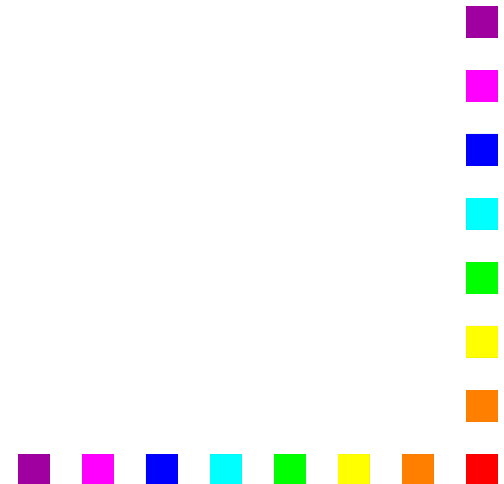
StudentID	StudentName	Course.
9 3 7 8 8 1 1	B i l l y D o w n e s	L M 0 2 1



9	3	3	4	5	6	7	F	r	a	n	k		C	u	r	t	a	i	n			L	M	0	5	1
9	3	8	3	7	1	5	T	h	o	m	a	s		H	e	a	l	y				L	M	0	6	8
9	3	4	7	2	9	2	T	o	n	y		O	'	B	r	i	a	n				L	M	0	5	1
9	3	7	8	8	1	1	B	i	l	l	y		D	o	w	n	e	s				L	M	0	2	1

READ StudentRecords

END-READ



How the READ works

StudentRecord

StudentID	StudentName	Course.
      	                  	    

HIGH-VALUES

9	3	3	4	5	6	7	F	r	a	n	k		C	u	r	t	a	i	n			L	M	0	5	1
9	3	8	3	7	1	5	T	h	o	m	a	s		H	e	a	l	y				L	M	0	6	8
9	3	4	7	2	9	2	T	o	n	y		O	'	B	r	i	a	n				L	M	0	5	1
9	3	7	8	8	1	1	B	i	l	l	y		D	o	w	n	e	s				L	M	0	2	1

EOF

AT END MOVE HIGH-VALUES TO StudentRecord



WRITE Syntax.

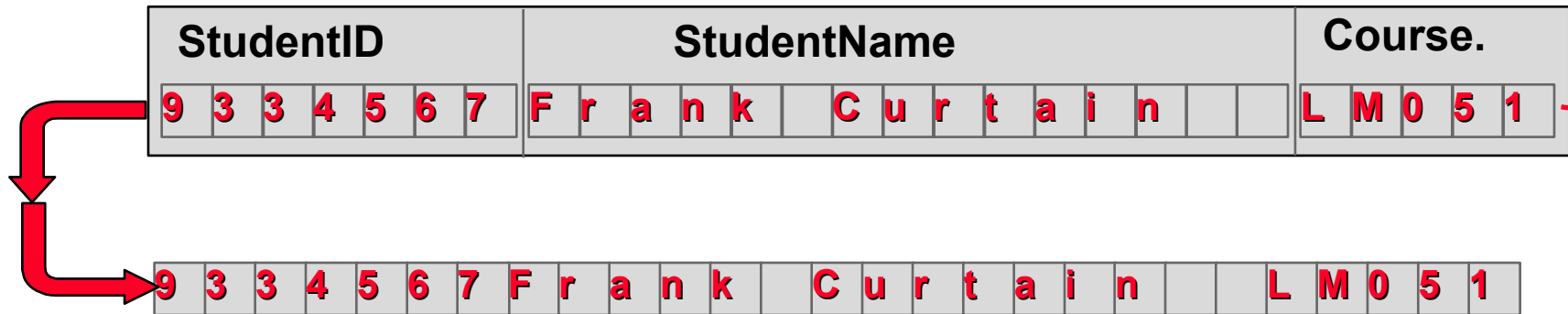
WRITE RecordName [FROM Identifier]
[{ BEFORE } ADVANCING { AdvanceNum [LINE] }
{ AFTER } MnemonicName
PAGE]]

To WRITE data to a file move the data to the record buffer (declared in the FD entry) and then **WRITE** the contents of record buffer to the file.



How the WRITE works

```
OPEN OUTPUT StudentFile.  
MOVE "9334567Frank Curtain LM051" TO StudentDetails.  
WRITE StudentDetails.  
  
MOVE "9383715Thomas Healy LM068" TO StudentDetails.  
WRITE StudentDetails.  
CLOSE StudentFile.  
STOP RUN.
```



How the WRITE works

```
OPEN OUTPUT StudentFile.  
MOVE "9334567Frank Curtain LM051" TO StudentDetails.  
WRITE StudentDetails.  
  
MOVE "9383715Thomas Healy LM068" TO StudentDetails.  
WRITE StudentDetails.  
CLOSE StudentFile.  
STOP RUN.
```

StudentID	StudentName	Course.
9 3 8 3 7 1 5	T h o m a s H e a l y	L M 0 6 8

9 3 3 4 5 6 7	F r a n k C u r t a i n	L M 0 5 1
9 3 8 3 7 1 5	T h o m a s H e a l y	L M 0 6 8


```
$ SET SOURCEFORMAT"FREE"
```

```
IDENTIFICATION DIVISION.
```

```
PROGRAM-ID. SeqWrite.
```

```
AUTHOR. Michael Coughlan.
```

```
ENVIRONMENT DIVISION.
```

```
INPUT-OUTPUT SECTION.
```

```
FILE-CONTROL.
```

```
    SELECT StudentFile ASSIGN TO "STUDENTS.DAT"
```

```
        ORGANIZATION IS LINE SEQUENTIAL.
```

```
DATA DIVISION.
```

```
FILE SECTION.
```

```
FD StudentFile.
```

```
01 StudentDetails.
```

```
    02 StudentId          PIC 9(7).
```

```
    02 StudentName.
```

```
        03 Surname       PIC X(8).
```

```
        03 Initials      PIC XX.
```

```
    02 DateOfBirth.
```

```
        03 YOBirth       PIC 9(2).
```

```
        03 MOBirth       PIC 9(2).
```

```
        03 DOBirth       PIC 9(2).
```

```
    02 CourseCode        PIC X(4).
```

```
    02 Grant             PIC 9(4).
```

```
    02 Gender            PIC X.
```

```
PROCEDURE DIVISION.
```

```
Begin.
```

```
    OPEN OUTPUT StudentFile.
```

```
    DISPLAY "Enter student details using template below.  Enter no data to end.".
```

```
    PERFORM GetStudentDetails.
```

```
    PERFORM UNTIL StudentDetails = SPACES
```

```
        WRITE StudentDetails
```

```
        PERFORM GetStudentDetails
```

```
    END-PERFORM.
```

```
    CLOSE StudentFile.
```

```
    STOP RUN.
```

```
GetStudentDetails.
```

```
    DISPLAY "NNNNNNNNSSSSSSSSIIYYMMDDCCCCGGGGGS".
```

```
    ACCEPT StudentDetails.
```

```
$ SET SOURCEFORMAT"FREE"
```

```
IDENTIFICATION DIVISION.
```

```
PROGRAM-ID. SeqRead.
```

```
AUTHOR. Michael Coughlan.
```

```
ENVIRONMENT DIVISION.
```

```
INPUT-OUTPUT SECTION.
```

```
FILE-CONTROL.
```

```
    SELECT StudentFile ASSIGN TO "STUDENTS.DAT"  
        ORGANIZATION IS LINE SEQUENTIAL.
```

```
DATA DIVISION.
```

```
FILE SECTION.
```

```
FD StudentFile.
```

```
01 StudentDetails.
```

```
    02 StudentId          PIC 9(7).
```

```
    02 StudentName.
```

```
        03 Surname        PIC X(8).
```

```
        03 Initials       PIC XX.
```

```
    02 DateOfBirth.
```

```
        03 YOBirth        PIC 9(2).
```

```
        03 MOBirth        PIC 9(2).
```

```
        03 DOBirth        PIC 9(2).
```

```
    02 CourseCode        PIC X(4).
```

```
    02 Grant              PIC 9(4).
```

```
    02 Gender             PIC X.
```

```
PROCEDURE DIVISION.
```

```
Begin.
```

```
    OPEN INPUT StudentFile
```

```
    READ StudentFile
```

```
        AT END MOVE HIGH-VALUES TO StudentDetails
```

```
    END-READ
```

```
    PERFORM UNTIL StudentDetails = HIGH-VALUES
```

```
        DISPLAY StudentId SPACE StudentName SPACE CourseCode
```

```
        READ StudentFile
```

```
            AT END MOVE HIGH-VALUES TO StudentDetails
```

```
    END-READ
```

```
    END-PERFORM
```

```
    CLOSE StudentFile
```

```
    STOP RUN.
```